

Implementación de Gestión de Usuarios

Actividad N°: 18 **Fecha:** 24/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar el módulo CRUD de usuarios, restringido al rol Jefe, aplicando las reglas de negocio definidas en la etapa de requerimientos (RF-09).

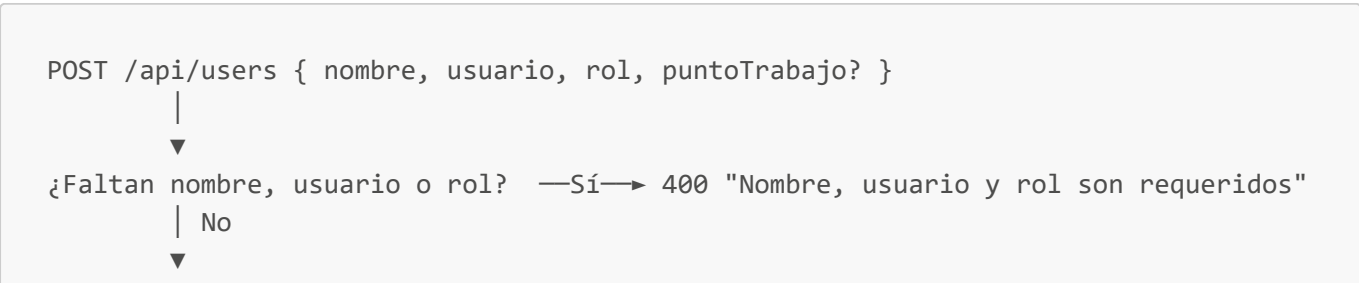
2. Endpoints implementados (src/routes/users.js + userController.js)

Método	Ruta	Acceso	Descripción
POST	/api/users	Jefe	Crear usuario nuevo
GET	/api/users	Jefe	Listar usuarios activos
PUT	/api/users/:id	Jefe	Actualizar usuario
DELETE	/api/users/:id	Jefe	Desactivar usuario (borrado lógico)

3. Reglas de negocio implementadas

- Contraseña inicial estándar:** todo usuario nuevo se crea con la contraseña definida en `DEFAULT_PASSWORD` y `primerAcceso: true`, forzando el cambio en el primer login (conecta con el módulo de autenticación del día anterior).
- Punto de trabajo condicional:** obligatorio si el rol es `staff`, no aplica si el rol es `jefe`.
- Unicidad de usuario:** se verifica explícitamente que el nombre de usuario no exista antes de crear la cuenta, devolviendo un error claro en caso de duplicado.
- Borrado lógico, no físico:** `deleteUser` no elimina el documento, marca `activo: false`, preservando el historial de auditoría asociado a ese usuario (los `AuditLog` referencian `usuario` por `_id`, y borrar el usuario físicamente rompería esa trazabilidad).
- Protección de la propia cuenta:** un usuario Jefe no puede desactivarse ni eliminarse a sí mismo, evitando bloqueos accidentales del único administrador activo.
- Listado solo de activos:** `getUsers` filtra `activo: true` para no mostrar cuentas dadas de baja en la gestión diaria.

4. Diagrama de creación de usuario



```
¿rol = staff y falta puntoTrabajo? —Sí→ 400 "Punto de trabajo es requerido"
  | No
  ▼
¿Usuario ya existe? —Sí→ 400 "El usuario ya existe"
  | No
  ▼
Crear con password = DEFAULT_PASSWORD, primerAcceso = true, creadoPor =
req.user._id
  |
  ▼
201 { usuario creado } (el hook pre-save hashea la contraseña automáticamente)
```

5. Frontend — `UsersPage.jsx`

- Tabla de usuarios activos con nombre, usuario, rol y punto de trabajo.
- Formulario de alta que muestra/oculta el campo "Punto de trabajo" dinámicamente según el rol seleccionado.
- Acciones de editar y desactivar, con confirmación previa (SweetAlert2) antes de desactivar una cuenta.

6. Conclusiones del día

El módulo de gestión de usuarios queda implementado con las validaciones de negocio necesarias (unicidad, punto de trabajo condicional, protección de autoeliminación) y con borrado lógico para preservar la integridad del historial de auditoría.

Observaciones: Sin observaciones.